BỘ TÀI LIỆU LẬP TRÌNH ESP32 – Share FREE

Biên Soạn: Kiến thức Điện Tử

Contact:

- Email: kienthucdientu.vn@gmail.com
- SDT: 079.809.4917
  - Github share tài liệu: https://github.com/KienthucDientu
  - Tiktok: https://www.tiktok.com/@kienthucdientu.vn?is_from_webapp=1&sender_device=pc
  - Youtube: https://www.youtube.com/@kienthucdientuvn
  - Fanfage: https://www.facebook.com/profile.php?id=61595485790496
  - Cộng đồng: https://www.facebook.com/groups/1397761335321670



- Chuyên cung cấp các giải pháp IOT, điều khiển từ xa, giám sát
- Thiết kế mạch điện tử PCB
- Share tài liệu FREE
- Cung cấp các khóa học về vi điều khiển

7.2 Lập trình ESP32 cơ bản với Arduino IDE

Việc chọn Arduino IDE mang lại hiệu quả vì nó có giao diện đơn giản, chỉ cần cài gói "ESP32 by Espressif" qua Boards Manager là sinh viên có thể bắt đầu lập trình ngay mà không phải lo lắng về thiết lập toolchain phức tạp, đồng thời hàng loạt thư viện và ví dụ mẫu (Wi-Fi, Bluetooth, sensor, MQTT…) đã sẵn sàng, giúp tiết kiệm thời gian thực hành cơ bản; khi cần nâng cao, người học vẫn có thể "nhúng" các hàm ESP-IDF bên trong môi trường Arduino hoặc chuyển sang PlatformIO/ESP-IDF để khám phá sâu hơn về FreeRTOS, OTA và các tính năng nâng cao của ESP32.

7.2.1 Chương trình ESP32 đầu tiên

Mọi chương trình học nên được khởi đầu một cách tuần tự và có hệ thống, đặc biệt trong giai đoạn đầu khi người học chưa có nhiều kiến thức hoặc kinh nghiệm về lĩnh vực này. Bài thực hành "Nhấp nháy LED (Blink)" được lựa chọn nhằm giúp sinh viên làm quen với những kiến thức cơ bản nhất. Thông qua bài này, sinh viên sẽ học cách biên dịch và nạp chương trình lên vi điều khiển ESP32, sử dụng các hàm và thư viện của Arduino để điều khiển chân GPIO, đồng thời hiểu được cấu trúc cơ bản của một chương trình Arduino với hai hàm quan trọng: setup() dùng để khởi tạo cấu hình và loop() dùng để thực thi lặp vô hạn.

Chương trình 7.1 Chương trình Blink đơn giản cho ESP32 (LED trên chân GPIO 2)

```
const int ledPin = 2;  // Chân GPIO nối LED void setup() {
  pinMode(ledPin, OUTPUT);  // Khai báo chân 2 là OUTPUT
}
void loop() {
  digitalWrite(ledPin, HIGH);  // Bật LED   delay(500);                // Chờ
500 ms   digitalWrite(ledPin, LOW);   // Tắt LED   delay(500);
// Chờ 500 ms
}
```

Nạp code này bằng Arduino IDE (chọn board "ESP32 Dev Module" và cổng COM tương ứng), ESP32 sẽ tự động nhấp nháy LED với chu kỳ 1 giây (500 ms ON, 500 ms OFF).

Lưu ý: Trong quá trình kết nối ESP32 và Máy tính nếu trong quá trình nạp chương trình không phát hiện được cổng COM hãy tìm và cài đặt "CP2102 Driver" hoặc "CH340
Driver" tương ứng với chip trên bo mạch ESP32 (tùy từng phiên bản ESP32 DevKit V1).

7.2.2 GPIO

Một trong những ngoại vi (peripheral) được sử dụng phổ biến và linh hoạt nhất trên vi điều khiển là GPIO (General Purpose Input Output – Ngõ vào/ra mục đích chung). GPIO thường được sử dụng để ghi (xuất) hoặc đọc trạng thái của một chân (pin).

GPIO chịu trách nhiệm điều khiển hoặc đọc trạng thái của một chân cụ thể trong thế giới số (digital world). Ví dụ, ngoại vi này thường được ứng dụng để tạo hiệu ứng nhấp nháy đèn LED (LED blinking) hoặc đọc tín hiệu từ một nút nhấn đơn giản (button). Có hai chế độ chính trong cấu hình GPIO:

- Chế độ ngõ vào – Input Mode: Ở chế độ này, GPIO sẽ nhận tín hiệu trạng thái số (digital state) từ một thiết bị bên ngoài. Thiết bị này có thể là một nút nhấn (button) hoặc công tắc (switch).
- Chế độ ngõ ra – Output Mode: Ở chế độ ngõ ra, GPIO sẽ xuất tín hiệu trạng thái số đến một thiết bị cụ thể. Ví dụ, ta có thể điều khiển một đèn LED (LED) bật hoặc tắt thông qua GPIO.

Cấu hình chế độ chân GPIO (pinMode)

Hàm pinMode() được sử dụng để cấu hình chế độ hoạt động của GPIO cho một chân (pin) cụ thể:

```
void pinMode(uint8_t pin, uint8_t mode);
```

- pin: số hiệu chân GPIO cần cấu hình.
- mode: thiết lập chế độ hoạt động.

Các chế độ sau được hỗ trợ cho đầu vào và đầu ra cơ bản (basic input and output):

- INPUT: thiết lập chân GPIO ở chế độ đầu vào không có điện trở kéo lên/kéo xuống nội bộ (input without pullup or pulldown), tức là ở trạng thái trở kháng cao (high impedance).
- OUTPUT: thiết lập chân GPIO ở chế độ đầu ra/đọc (output/read mode).
- INPUT_PULLDOWN: thiết lập chân GPIO ở chế độ đầu vào với điện trở kéo xuống nội bộ (input with internal pulldown).
- INPUT_PULLUP: thiết lập chân GPIO ở chế độ đầu vào với điện trở kéo lên nội bộ (input with internal pullup

Các dòng SoC ESP32 (ESP32 SoC families) hỗ trợ điện trở kéo lên (pullup) và kéo xuống (pulldown) nội bộ thông qua một điện trở 45 kΩ (45kΩ resistor), có thể được kích hoạt khi cấu hình chân GPIO ở chế độ INPUT. Nếu không cấu hình rõ ràng chế độ pullup hoặc pulldown, chân GPIO sẽ duy trì ở trạng thái trở kháng cao (high impedance), tức là không có mức logic xác định.

Hàm digitalWrite

Hàm digitalWrite được dùng để thiết lập trạng thái của chân GPIO được chọn ở mức HIGH (mức cao) hoặc LOW (mức thấp). Lưu ý: Hàm này chỉ hoạt động khi chân GPIO đã được cấu hình ở chế độ OUTPUT thông qua hàm pinMode.

```
void digitalWrite(uint8_t pin, uint8_t val);
```

Tham số của hàm digitalWrite(pin, val)

- pin: xác định số hiệu chân GPIO mà ta muốn điều khiển.
- val: thiết lập trạng thái logic đầu ra của chân GPIO: HIGH – mức cao ; LOW – mức thấp (0V)

Hàm digitalRead

Dùng để đọc trạng thái logic (HIGH hoặc LOW) của chân GPIO đã được cấu hình ở chế độ INPUT

```
int digitalRead(uint8_t pin);
```

- pin: chọn số chân GPIO cần đọc.
- Giá trị trả về: HIGH hoặc LOW.

Ngắt (Interrupts)

ESP32 hỗ trợ ngắt (interrupts) trên các chân GPIO.

Hàm attachInterrupt

Dùng để gắn hàm xử lý ngắt (interrupt handler) cho một chân GPIO xác định.

```
attachInterrupt(uint8_t pin, voidFuncPtr handler, int mode);
```

- pin: số chân GPIO.
- handler: hàm xử lý khi xảy ra ngắt.
- mode: chế độ kích hoạt ngắt.

Các chế độ ngắt hỗ trợ (mode):

- DISABLED: vô hiệu hóa ngắt
- RISING: ngắt khi tín hiệu lên mức cao
- FALLING: ngắt khi tín hiệu xuống mức thấp
- CHANGE: ngắt khi có thay đổi mức
- ONLOW: ngắt khi mức thấp
- ONHIGH: ngắt khi mức cao
- ONLOW_WE: giống ONLOW, hỗ trợ wake-up
- ONHIGH_WE: giống ONHIGH, hỗ trợ wake-up

Hàm detachInterrupt

Dùng để gỡ bỏ ngắt khỏi một chân GPIO đã được gắn trước đó.

```
detachInterrupt(uint8_t pin);
```

Chương trình 7.2

 Mỗi lần nhấn nút, LED chuyển trạng thái (bật → tắt hoặc tắt → bật). Dùng chân 2 làm nút nhấn kiểu PULLUP, chân 12 điều khiển LED.

Chưa có xử lý chống dội (debounce) nên có thể nháy LED khi nhấn nhanh

```
#define LED    12
#define BUTTON 2


uint8_t stateLED = 0;   void setup()
{
    pinMode(LED, OUTPUT);      pinMode(BUTTON,INPUT_PULLUP);
 }
 void loop() {
   if(!digitalRead(BUTTON)){       stateLED =
stateLED^1;      digitalWrite(LED,stateLED);
   }
```

```
}
```

Chương trình 7.3.

Chương trình sử dụng ngắt (interrupt) để đếm số lần nhấn của hai nút bấm:

- Nút 1 (GPIO 23) dùng attachInterruptArg() để truyền tham chiếu cấu trúc.
- Nút 2 (GPIO 18) dùng attachInterrupt() thông thường.

Mỗi lần nhấn, số lần nhấn (numberKeyPresses) tăng và cờ pressed bật. Trong vòng lặp loop(), nếu có nút được nhấn, chương trình in ra số lần nhấn qua Serial. Sau 10 giây, chương trình gỡ bỏ ngắt của nút 1.

```
#include <Arduino.h> struct
Button {   const uint8_t PIN;
uint32_t numberKeyPresses;   bool
pressed;
};
Button button1 = {23, 0, false}; Button button2 =
{18, 0, false}; void ARDUINO_ISR_ATTR isr(void
*arg) {   Button *s = static_cast<Button *>(arg);   s-
>numberKeyPresses += 1;   s->pressed = true;
}
void ARDUINO_ISR_ATTR isr() {
button2.numberKeyPresses += 1;   button2.pressed = true;
}
void setup() {
  Serial.begin(115200);
  pinMode(button1.PIN, INPUT_PULLUP);
  attachInterruptArg(button1.PIN, isr, &button1, FALLING);   pinMode(button2.PIN,
INPUT_PULLUP);   attachInterrupt(button2.PIN, isr, FALLING);
```

```
}
void loop() {
  if (button1.pressed) {
    Serial.printf("Button 1 has been pressed %lu times\n", button1.numberKeyPresses);
    button1.pressed = false;
  }
  if (button2.pressed) {
    Serial.printf("Button 2 has been pressed %lu times\n", button2.numberKeyPresses);
    button2.pressed = false;
  }
  static uint32_t lastMillis = 0;   if (millis() -
lastMillis > 10000) {     lastMillis = millis();
detachInterrupt(button1.PIN);
  }
}
```

7.2.3 ADC – Analog to Digital Converter

ESP32 hỗ trợ bộ chuyển đổi tín hiệu analog sang số (ADC) với độ phân giải mặc định 12 bit, chia thành hai khối: ADC1 và ADC2, mỗi khối gồm nhiều kênh đầu vào analog.

```
int analogRead(uint8_t pin);
```

Tham số

- pin: Số hiệu chân GPIO hỗ trợ ADC. Phải là một trong các chân được phép đo analog (Xem Sơ đồ chân ở hình bên trên)

Giá trị trả về

- Trả về một giá trị nguyên dương từ 0 đến 4095 (12 bit), tương ứng với mức điện áp từ 0V đến Vref

Lưu ý: ADC2 bị giới hạn khi dùng Wi-Fi → nên ưu tiên sử dụng ADC1.

Chương trình 7.4. Đọc giá trị ADC ở chân 34 và hiển thị Serial

```
#define SENSOR_PIN 34  // ADC1_CH6
```

```
void setup() {
  Serial.begin(115200);   pinMode(SENSOR_PIN,
INPUT);
}
void loop() {
  int adcValue = analogRead(SENSOR_PIN);   Serial.printf("ADC Value:
%d\n", adcValue);   delay(500);
}
```

7.2.4 LED Control (LEDC) – Bộ điều khiển PWM tích hợp

Bộ ngoại vi LEDC (LED Control) trên ESP32 được thiết kế chủ yếu để điều khiển độ sáng của đèn LED thông qua kỹ thuật PWM (Pulse Width Modulation – điều chế độ rộng xung). Ngoài ra, LEDC cũng có thể được sử dụng để tạo tín hiệu PWM cho các mục đích khác, như điều khiển tốc độ động cơ, tín hiệu servo, hoặc phát xung điều khiển.

Các SoC ESP32 hỗ trợ từ 6 đến 16 kênh PWM độc lập (Đối với ESP32 của devkit V1 trên là 16 chân), cho phép tạo dạng sóng riêng biệt, thích hợp để điều khiển thiết bị như LED RGB. analogWrite

Hàm analogWrite() được dùng để ghi giá trị analog (dạng sóng PWM) lên một chân GPIO.Tương thích với hàm analogWrite() của Arduino với các đặc điểm sau:

- Độ phân giải PWM: 8 bit (giá trị value từ 0 đến 255).
- Tần số PWM: 1 kHz (1000 Hz).

 Độ rộng xung (Duty cycle): value / 255 * 100%

```
void analogWrite(uint8_t pin, int value);
```

- pin: chọn chân GPIO.
- value: thiết lập chu kỳ nhiệm vụ (duty cycle) của PWM, từ 0 (tắt hoàn toàn) đến 255 (bật liên tục).

Nếu muốn tạo PWM nâng cao thì đến với phần sau đây ledcAttach

Hàm ledcAttach() dùng để cấu hình chân LEDC với tần số và độ phân giải PWM chỉ định. Kênh LEDC sẽ được chọn tự động.

```
bool ledcAttach(uint8_t pin, uint32_t freq, uint8_t resolution);
```

- pin: chân GPIO cần phát PWM.
- freq: tần số PWM (Hz).
- resolution: độ phân giải PWM (số bit) ledcWrite

Hàm ledcWrite() dùng để thiết lập chu kỳ nhiệm vụ (duty cycle) cho chân LEDC đã cấu hình.

```
bool ledcWrite(uint8_t pin, uint32_t duty);
```

- pin: chân GPIO đã gán LEDC.
- duty: giá trị duty cần đặt (phụ thuộc vào độ phân giải đã chọn).

ESP32 sử dụng bộ ngoại vi LEDC (LED Control) để tạo tín hiệu PWM (Pulse Width Modulation) với tần số và độ phân giải có thể tùy chỉnh. Tuy nhiên, tần số (freq) và độ phân giải (resolution) không thể cấu hình một cách độc lập, mà phụ thuộc lẫn nhau, giới hạn bởi clock nguồn (APB clock, 80 MHz).

Độ phân giải PWM: từ 1 đến 16 bit, tương ứng với $2^1 = 2$ mức đến $2^{16} = 65536$ mức duty cycle. Tần số PWM có thể cấu hình lên đến 40 MHz, nhưng không đồng thời đạt được với độ phân giải cao. Để tín hiệu PWM hoạt động đúng, công thức sau phải được thỏa mãn $Freq \times 2^{Res} \times Clock$

Trong đó:

- PWM_freq: tần số PWM mong muốn (Hz)
- resolution: độ phân giải (bit)
- Clock_source: xung nhịp LEDC, mặc định là 80 MHz trên ESP32

Dưới đây là ví dụ cấu hình PWM với tần số 30 kHz và tính toán độ phân giải tối đa có thể đạt được trên ESP32:

$$2^{res} \times \frac{Clock}{Freq} \times \frac{80000000}{30000} \times 2666.67$$

Lấy $\log_2$ để có $res \times \log 2666.67_2 \times 11.38$. Vậy, độ phân giải tối đa cho xung PWM đó là 11 bit.

Chương trình 7.5. Tạo xung PWM có độ động 50% tần số 30 KHz

```
#define PWM_PIN    25      // GPIO dùng làm đầu ra PWM
#define PWM_FREQ   30000   // Tần số 30kHz
```

```
#define PWM_RES    11        // Độ phân giải 11 bit (0–2047) void setup() {
  Serial.begin(115200);
  bool ok = ledcAttach(PWM_PIN, PWM_FREQ, PWM_RES);   if
(ok) {
    // Viết duty 50%
    uint32_t duty =  50.0 / 100.0 * 2048.0;     ledcWrite(PWM_PIN, duty);
    Serial.println("PWM configured successfully.");
  } else {
    Serial.println("Failed to configure PWM.");
  } }
void loop() {}
```

7.2.5 Bluetooth

ESP32 là một vi điều khiển tích hợp Wi-Fi và Bluetooth của hãng Espressif Systems, nổi bật với khả năng kết nối không dây đa dạng. Trong đó, Bluetooth là một tính năng quan trọng cho các ứng dụng IoT cá nhân, truyền dữ liệu tầm gần, và giao tiếp với điện thoại thông minh. ESP32 có thể chạy đồng thời cả Classic Bluetooth và BLE (Dualmode), nhưng việc này cần tài nguyên RAM phù hợp. Các tính năng như sau:

- Bluetooth v4.2 (trên hầu hết các dòng)
- Tốc độ truyền dữ liệu BLE lên đến 2 Mbps (BLE 4.2)
- Khả năng đóng vai trò Central và Peripheral trong BLE.
- Hỗ trợ chuẩn GATT server/client
- Tương thích với các framework phổ biến: ESP-IDF (native), Arduino (thư viện BLEDevice, BluetoothSerial).

Lưu ý: Bluetooth được hỗ trợ hầu hết trên các dòng của ESP32, ngoại trừ ESP32-S2 không hỗ trợ Bluetooth; ESP32-S3, C3, C6 và H2 chỉ hỗ trợ BLE, không hỗ trợ Bluetooth Classic.

Chương trình 7.6.
Chương trình này biến ESP32 thành thiết bị BLE Central, có nhiệm vụ quét và liệt kê các thiết bị BLE xung quanh đang phát tín hiệu (advertising).

```
// This example code is in the Public Domain (or CC0 licensed, at your option.)
// By Evandro Copercini - 2018
//
// This example creates a bridge between Serial and Classical Bluetooth (SPP)
// and also demonstrate that SerialBT have the same functionalities of a normal Serial
// Note: Pairing is authenticated automatically by this device
#include "BluetoothSerial.h"

String device_name = "ESP32-BT-Slave";
// Check if Bluetooth is available
#if !defined(CONFIG_BT_ENABLED) || !defined(CONFIG_BLUEDROID_ENABLED)
#error Bluetooth is not enabled! Please run `make menuconfig` to and enable it
#endif

// Check Serial Port Profile
#if !defined(CONFIG_BT_SPP_ENABLED)
#error Serial Port Profile for Bluetooth is not available or not enabled. It is only available for the
 ESP32 chip.
#endif

BluetoothSerial SerialBT;

void setup() {
 Serial.begin(115200);
 SerialBT.begin(device_name);  //Bluetooth device name
 //SerialBT.deleteAllBondedDevices(); // Uncomment this to delete paired devices; Must be called
 after begin
 Serial.printf("The device with name \"%s\" is started.\nNow you can pair it with Bluetooth!\n",
 device_name.c_str());
}
void loop() {
 if (Serial.available()) {
  SerialBT.write(Serial.read());
 }
 if (SerialBT.available()) {
  Serial.write(SerialBT.read());
 }
 delay(20);
}
```

Chương trình 7.7. Chương trình này biến ESP32 thành thiết bị BLE Central, có nhiệm vụ quét và liệt kê các thiết bị BLE xung quanh đang phát tín hiệu (advertising).

```
/*
  Based on Neil Kolban example for IDF: https://github.com/nkolban/esp32snippets/blob/master/cpp_utils/tests/BLE%20Tests/SampleScan.cpp
Ported to Arduino ESP32 by Evandro Copercini
*/

#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEScan.h>
#include <BLEAdvertisedDevice.h>

int scanTime = 5;  //In seconds
BLEScan *pBLEScan;

class MyAdvertisedDeviceCallbacks : public BLEAdvertisedDeviceCallbacks {   void onResult(BLEAdvertisedDevice advertisedDevice) {
                    Serial.printf("Advertised Device: %s \n", advertisedDevice.toString().c_str());
  }
};
void setup() {
  Serial.begin(115200);
  Serial.println("Scanning...");

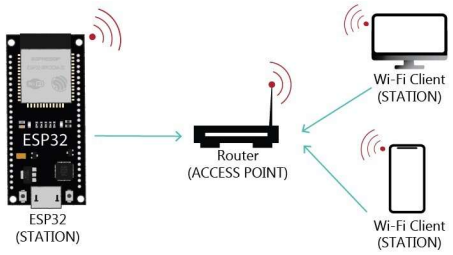
  BLEDevice::init("");
  pBLEScan = BLEDevice::getScan();  //create new scan
                                                    pBLEScan->setAdvertisedDeviceCallbacks(new MyAdvertisedDeviceCallbacks());   pBLEScan->setActiveScan(true);  //active scan uses more power, but get
  results faster
  pBLEScan->setInterval(100);
  pBLEScan->setWindow(99);  // less or equal setInterval value }
void loop() {
  // put your main code here, to run repeatedly:
  BLEScanResults *foundDevices = pBLEScan->start(scanTime, false);
  Serial.print("Devices found: ");
  Serial.println(foundDevices->getCount());
  Serial.println("Scan done!");   pBLEScan->clearResults();  // delete results fromBLEScan buffer to release memory
  delay(2000);
}
```

7.2.6 Wi-Fi

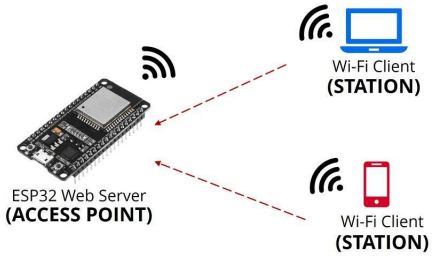
ESP32 là một vi điều khiển tích hợp Wi-Fi và Bluetooth, được thiết kế cho các ứng dụng IoT với khả năng kết nối mạng mạnh mẽ, chi phí thấp và dễ lập trình. Trong đó, Wi-Fi là một thành phần cốt lõi, giúp thiết bị kết nối với Internet để truyền, nhận và đồng bộ dữ liệu với các dịch vụ đám mây hoặc thiết bị khác.

ESP32 có khả năng kết nối tới Wi-Fi ở băng tần 2.4 Ghz (ESP32 – C5 là dòng duy nhất tính đến hiện tại có thể kết nối tới sóng 5.0Ghz). Các chế độ Wi-Fi đối với ESP32 bao gồm:

- Chế độ trạm (Wi-Fi Station): Cho phép ESP32 kết nối tới các Router. STA mode được sử dụng nhiều, giúp thiết bị kết nối mạng Wi-Fi cục bộ, có internet để kết nối đến Server và gửi dữ liệu
- Chế độ AP (Access point): cho phép các clients khác kết nối vào. AP mode được sử dụng nhiều trong việc biến ESP32 trở thành một web server, cho phép các thiết bị khác kết nối nhưng không có Internet



Hình 7.5. ESP32 chế độ Station



Hình 7.6. ESP32 chế độ AP

Chương trình 7.8. Chương trình này có chức năng quét qua các mạng Wi-Fi có sẵn trong phạm vi kết nối của ESP32, sau đó hiển thị thông tin chi tiết của từng mạng lên Serial Monitor

```
/*
*    This sketch demonstrates how to scan WiFi networks.
*    The API is based on the Arduino WiFi Shield library, but has significant changes as newer WiFi
     functions are supported.
*    E.g. the return value of `encryptionType()` different because more modern encryption is
     supported.
 */
#include "WiFi.h" void
setup() {
  Serial.begin(115200);
  // Set WiFi to station mode and disconnect from an AP if it was previously connected.
  WiFi.mode(WIFI_STA);
WiFi.disconnect();   delay(100);
  Serial.println("Setup done");
}
void loop() {
  Serial.println("Scan start");

  // WiFi.scanNetworks will return the number of networks found.
  int n = WiFi.scanNetworks();   Serial.println("Scan done");
  if (n == 0) {
    Serial.println("no networks found");
  } else {
    Serial.print(n);
    Serial.println(" networks found");
    Serial.println("Nr | SSID                             | RSSI | CH | Encryption");     for (int i = 0; i < n;
++i) {
      // Print SSID and RSSI for each network found
      Serial.printf("%2d", i + 1);
      Serial.print(" | ");
      Serial.printf("%-32.32s", WiFi.SSID(i).c_str());
      Serial.print(" | ");
      Serial.printf("%4ld", WiFi.RSSI(i));
      Serial.print(" | ");
      Serial.printf("%2ld", WiFi.channel(i));       Serial.print(" | ");
      switch (WiFi.encryptionType(i)) {
        case WIFI_AUTH_OPEN:          Serial.print("open"); break;        case
WIFI_AUTH_WEP:          Serial.print("WEP"); break;        case WIFI_AUTH_WPA_PSK:
Serial.print("WPA"); break;
```

```
      case  WIFI_AUTH_WPA2_PSK:              Serial.print("WPA2");  break;              case
WIFI_AUTH_WPA_WPA2_PSK:        Serial.print("WPA+WPA2");  break;              case
WIFI_AUTH_WPA2_ENTERPRISE:  Serial.print("WPA2-EAP");  break;              case
WIFI_AUTH_WPA3_PSK:              Serial.print("WPA3");  break;              case
WIFI_AUTH_WPA2_WPA3_PSK:       Serial.print("WPA2+WPA3");  break;              case
WIFI_AUTH_WAPI_PSK:              Serial.print("WAPI");  break;              default:
Serial.print("unknown");      }
    Serial.println();      delay(10);
   }
  }
  Serial.println("");
  // Delete the scan result to free memory for code below.
  WiFi.scanDelete();
  // Wait a bit before scanning again.
  delay(5000);
}
```

Kết quả đầu ra mong đợi của Chương trình 7.8

```
Scan start
Scan done
4 networks found
Nr | SSID                     | RSSI | CH | Encryption
1  | MyHomeWiFi               |  -67 |  6 | WPA2
2  | Coffee_Shop              |  -78 | 11 | open
3  | Office_AP                |  -70 |  1 | WPA2+WPA3
4  | GuestNet                 |  -82 |  9 | WPA
```

Chương trình 7.9:

Chương trình này biến ESP32 thành một Wi-Fi Access Point (AP), tức là nó tự phát ra một mạng Wi-Fi riêng, cho phép các thiết bị khác (như điện thoại, laptop) kết nối trực tiếp đến ESP32.

```
#include <WiFi.h> const char* ssid =
"LOCA"; const char* password =
"12345678";
void setup() {
  Serial.begin(115200);
  delay(10);
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
}
void loop() {}
```

Ví dụ đầu ra:

```
Configuring access point...AP IP address: 192.168.4.1
Stations connected = 0
Stations connected = 1
Stations connected = 2
```

ESP32 có khả năng cho phép các Client kết nối vào. Tối đa 10 STA (Do giới hạn về RAM) . Địa chỉ IP mặc định của ESP32 trong chế độ này là 192.168.4.1. Nếu muốn thay đổi thì ta sử dụng WiFi.softAPConfig thay vì WiFi.softAP

```
IPAddress local_IP(192,168,4,22);
IPAddress gateway(192,168,4,9);
IPAddress subnet(255,255,255,0);
WiFi.softAPConfig(local_IP, gateway, subnet)
```

7.2.7 HTTP

Lập trình ESP32 với HTTP là một kỹ năng cốt lõi trong IoT, vì nó giúp thiết bị giao tiếp với Web Server, Cloud, hoặc người dùng qua mạng Internet. ESP32 có thể thực hiện các vai trò sau qua HTTP:

Bảng 7.2. Vai trò của ESP32 qua HTTP

Vai trò	Giải thích	Ví dụ

HTTP Client	Gửi request đến server bên ngoài	Gửi dữ liệu cảm biến lên ThingSpeak, Firebase, v.v.
HTTP Server	Tự tạo trang web cho người dùng truy cập và điều khiển	Điều khiển LED từ điện thoại qua trình duyệt

Vai trò	Giải thích	Ví dụ
REST API Client/Server	Trao đổi dữ liệu JSON qua HTTP	Nhận và gửi dữ liệu ở dạng RESTful (GET, POST, PUT, DELETE)

Trước khi bắt đầu với HTTP, ta hãy nhắc lại một số khái nghiệm quan trọng:

URL

URL hay Uniform Resource Locator là một khái niệm quan trọng trong HTTP, các thành phần như sau:

http://server.com:8080/path/to/log.php?nhiet_do=30&do_am=80#test

Bảng 7.3. Các thành phần của URL

Thành phần	Ví dụ	Mô tả
Scheme	http	Giao thức sử dụng, như http, https, ftp, v.v.
Host	server.com	Tên miền hoặc địa chỉ IP của máy chủ
Port	8080	(Tuỳ chọn) Cổng kết nối, mặc định 80 cho HTTP và 443 cho HTTPS
Path	/path/to/log.php	Đường dẫn tới tài nguyên hoặc tập tin trên máy chủ
Query	?nhiet_do=30&do_am=80	(Tuỳ chọn) Các tham số gửi lên server, thường ở dạng key=value
Fragment	#test	(Tuỳ chọn) Thẻ đánh dấu nội dung trong tài nguyên (thường dùng phía client)

JSON

JSON (JavaScript Object Notation) là một định dạng dữ liệu dạng văn bản, nhẹ và dễ đọc/ghi, được sử dụng để trao đổi dữ liệu giữa client và server, đặc biệt trong các ứng dụng web và IoT. Các đặc điểm của JSON như sau:

- Dạng văn bản: Dữ liệu có thể đọc được bằng mắt, dễ debug
- Định dạng key-value: Mỗi giá trị gắn với một tên khóa
- Phổ biến trên Web, REST API, IoT: JSON là chuẩn mặc định cho nhiều hệ thống cloud và dịch vụ IoT
- Tương thích đa nền tảng: Hỗ trợ tốt trong hầu hết các ngôn ngữ lập trình: C, Python, JavaScript, Arduino, … Cấu trúc cơ bản của JSON:

```
{
  "nhiet_do": 30.5,
  "do_am": 82,
  "trang_thai": "OK"
}
```

Chương trình 7.10
Truy vấn giá Bitcoin (BTC) theo thời gian thực bằng HTTP GET request, và xử lý dữ liệu JSON trả về từ một API công khai.

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>  //https://github.com/bblanchon/ArduinoJson const char* ssid =
"*****"; const char* password = "*****"; const char* host = "https://min-
api.cryptocompare.com/data/price?fsym=BTC&tsyms=USD";

void getBitcoin() {
  HTTPClient http;
  Serial.print("connecting to "); Serial.println(host);   http.begin(host);
int httpCode = http.GET();   if (httpCode == HTTP_CODE_OK) {
    String payload = http.getString();
    Serial.println(payload);

    JsonDocument doc;
    DeserializationError error = deserializeJson(doc, payload);     if (error) {
      Serial.print(F("deserializeJson() failed: "));
      Serial.println(error.f_str());       return;
    }
    double priceUSD = doc["USD"];
    Serial.println(priceUSD);
  }
  http.end();
}
void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
```

```
  while (WiFi.status() != WL_CONNECTED) {     delay(500);
    Serial.print(".");
  }
}
void loop() {
  if (WiFi.status() == WL_CONNECTED) {     getBitcoin();
  }
  delay(5000);
}
```

Đầu ra ví dụ như sau:

```
connecting to https://min-api.cryptocompare.com/data/price?fsym=BTC&tsyms=USD
{"USD":67345.12}
67345.12
```

Giải thích chương trình:

- ESP32 kết nối với mạng Wi-Fi để có thể truy cập Internet.
- Sử dụng thư viện HTTPClient.h, ESP32 gửi yêu cầu GET đến địa chỉ API https://min-api.cryptocompare.com/data/price?fsym=BTC&tsyms=USD
- Server trả về một chuỗi JSON đơn giản như: { "USD": 67345.12 } và ESP32 sử dụng thư viện ArduinoJson.h để phân tích (deserialize) chuỗi JSON này, trích xuất giá trị của khóa "USD" rồi in ra Serial Monitor.

Chương trình 7.11 sau đây sẽ sử dụng HTTP PUT để gửi dữ liệu lên Server

Chương trình 7.11.1
Tạo một server bằng Flask trên python

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# Biến lưu dữ liệu tạm thời device_data = {}

# GET: Xem dữ liệu hiện tại
@app.route('/api/device/esp32', methods=['GET']) def get_data():
```

```
    return jsonify(device_data)

# PUT: Cập nhật dữ liệu từ ESP32
@app.route('/api/device/esp32',          methods=['PUT'])          def
update_data():
    global device_data     if
request.is_json:
        device_data = request.get_json()          return jsonify({"message": "Data updated", "data":
device_data}),
200
else:
        return jsonify({"error": "Invalid JSON"}), 400
 if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Khi chạy chương trình Python đó, ta sẽ có địa chỉ server là 192.168.1.53:5000 và địa chỉ server lắng nghe ESP32 là 192.168.1.53:5000/api/device/esp32

```
 * Serving Flask app 'myserver'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.1.53:5000
```

Chương trình 7.11.2: Chương trình ESP32 sử dụng HTTP PUT để ghi dữ liệu lên Server 192.168.1.53:5000

```
#include <WiFi.h>
#include <HTTPClient.h> #include
<ArduinoJson.h> const char* ssid =
"****"; const char* password =
"****";
const char* serverUrl = "http://192.168.1.53:5000/api/device/esp32";   void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {     delay(500);
```

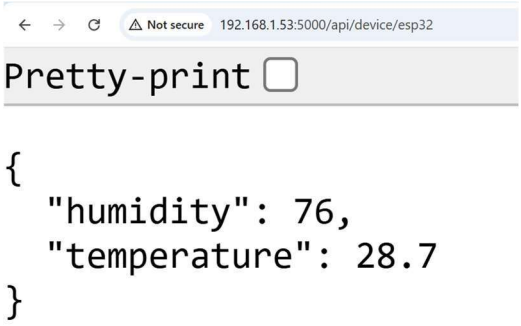
```
    Serial.print(".");
  }
  Serial.println("\nConnected to WiFi");   sendPutRequest();
}
void sendPutRequest() {
  if (WiFi.status() == WL_CONNECTED) {
HTTPClient http;     http.begin(serverUrl);
    http.addHeader("Content-Type", "application/json");
    // Tạo dữ liệu JSON
    StaticJsonDocument<200> doc;
doc["temperature"] = 28.7;
doc["humidity"] = 76;     String jsonBody;
    serializeJson(doc, jsonBody);     // Gửi PUT
request với body là JSON     int httpCode =
http.PUT(jsonBody);
    // Phản hồi
    Serial.print("HTTP Response code: ");
    Serial.println(httpCode);     if (httpCode
> 0) {
      String response = http.getString();
      Serial.println("Server response:");
      Serial.println(response);
    }
    http.end();
  } else {
    Serial.println("WiFi not connected");   }
}  void loop() {}
```

Khi chạy chương trình, ta sẽ nhận được:

```
.....
Connected to WiFi
HTTP Response code: 200 Server response:
{
  "data": {
    "humidity": 76,
    "temperature": 28.7
```

```
  },
  "message": "Data updated"
}
```

Ta cũng có thể sử dụng trình duyệt vào 192.168.1.53:5000/api/device/esp32 để xem



Hình 7.7. Dữ liệu dưới dạng JSON trên trình duyệt

7.2.8 Webserver

ESP32 có thể hoạt động như một máy chủ web (Web Server) – tức là:

- Tạo ra một trang web nội bộ (giao diện điều khiển hoặc hiển thị).
- Cho phép người dùng truy cập bằng trình duyệt thông qua địa chỉ IP.
- Có thể dùng để hiển thị dữ liệu cảm biến, bật/tắt thiết bị, hoặc nhập cấu hình.

Trước tiên, ta nhắc lại một số khái niệm

HTML (HyperText Markup Language) là ngôn ngữ đánh dấu dùng để tạo và cấu trúc nội dung của một trang web. Trong các ứng dụng ESP32 Web Server, HTML được gửi từ ESP32 đến trình duyệt, cho phép người dùng tương tác với thiết bị thông qua giao diện web. Cấu trúc cơ bản của một tài liệu HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>ESP32 Web Page</title>
</head>
<body>
  <h1>Welcome to ESP32 Web Server</h1>
  <p>Click to control the LED:</p>
  <a href="/on">Turn ON</a>
  <br>
  <a href="/off">Turn OFF</a>
</body>
```

```
</html>
```

Giải thích:

- <!DOCTYPE html>: Khai báo kiểu tài liệu HTML5.
- <html>: Bắt đầu tài liệu HTML.
- <head>: Chứa thông tin như tiêu đề trang.
- <body>: Chứa nội dung chính hiển thị trên trình duyệt.
- Thẻ <h1>, <p>, <a>: Các phần tử cơ bản của văn bản, đoạn, liên kết...

Javascript

- Javascript là một ngôn ngữ được thiết kế chủ yếu để thêm tương tác vào các trang Web, và tạo ra các ứng dụng Web.
- Javscript có thể nhúng vào HTML bằng tag <script> và thuộc tính type

Chương trình Javscript trong HTML

```
<!DOCTYPE html>
<html>
<head>
<title>My first JavaScript page</title>
</head>
<body>
<script type="text/javascript">
// chương trình Javascript nên được viết ở đây
// ngay trước </body>
</script>
</body>
</html>
```

CSS

- CSS là từ viết tắt của Cascading Style Sheets, là một ngôn ngữ được thiết kế để xử lý giao diện Web, giúp các trang Web được đẹp hơn
- CSS có thể kiểm soát được màu sắc của văn bản, phong chữ, kích cỡ chữ, khoảng cách giữa các đoạn văn, hình nền hoặc màu nền, và nhiều hiệu ứng khác.

- Các cách sử dụng CCS:

  - Inline:

```
<p style="color:red;">Cảnh báo!</p>
```

  - Internal:

```
<style> body { background-color: #f0f0f0; font-family: Arial; } h1 { color: blue; }
</style>
```

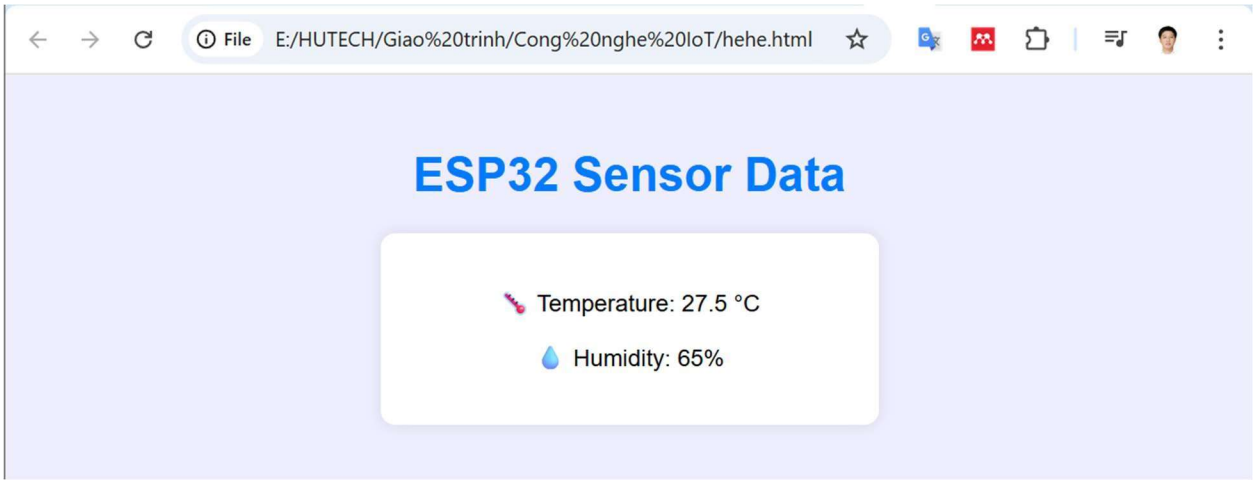
Bảng 7.4. Các thuộc tính phổ biến

Thuộc tính	Chức năng	Ví dụ
color	Màu chữ	color: red;
background-color	Màu nền	background-color: #f2f2f2;
font-size	Cỡ chữ	font-size: 20px;
text-align	Căn lề chữ	text-align: center;
margin, padding	Khoảng cách ngoài/trong	margin: 10px;
border	Viền	border: 1px solid black;
display, flex, grid	Bố cục nâng cao	display: flex;

Xét ví dụ sau:

```
<!DOCTYPE html>
<html>
<head>
  <title>ESP32 Sensor</title>
  <style>    body {      background-color: #eef;
font-family: Arial, sans-serif;      text-align:
center;      margin-top: 50px;
    }    h1 {      color:
#007BFF;
    }
    .data-box {      background-color:
#fff;      padding: 20px;      margin:
20px auto;      border-radius: 10px;
width: 300px;
    box-shadow: 0 0 10px rgba(0,0,0,0.1);
  }
 </style>
</head>
```

```
<body>
  <h1>ESP32 Sensor Data</h1>
  <div class="data-box">
    <p> 🌡 Temperature: 27.5 °C</p>
    <p> 💧 Humidity: 65%</p>
  </div>
</body>
</html>
```



Hình 7.8. Trang web với CSS

Chương trình 7.12
ESP32 đóng vai trò là webserver điều khiển ON/OFF đèn LED D2

```
#include <WiFi.h> #include <WebServer.h>
const char* ssid = "ESP32-AP"; const char*
password = "12345678"; WebServer
server(80);
const int ledPin = 2;          // GPIO2 (D2 trên nhiều board) bool ledState = false;          //
Lưu trạng thái LED
// Tạo HTML kèm trạng thái LED
String htmlPage() {
  String stateText = ledState ? "ON" : "OFF";

  String html = R"rawliteral(
<!DOCTYPE html>
<html>
```

```
<head>
  <title>ESP32 LED Control</title>
  <style>
    body { font-family: Arial; text-align: center; margin-top: 50px; }     .status { font-size:
1.5em; margin-bottom: 20px; }
    a.button {
      display: inline-block;       padding:
15px 25px;       font-size: 20px;
background-color: #007BFF;       color:
white;       text-decoration: none;
border-radius: 8px;       margin: 10px;
    }
  </style>
</head>
<body>
  <h1>ESP32 Access Point Web Server</h1>
  <p class="status">LED is now: <strong>)rawliteral";

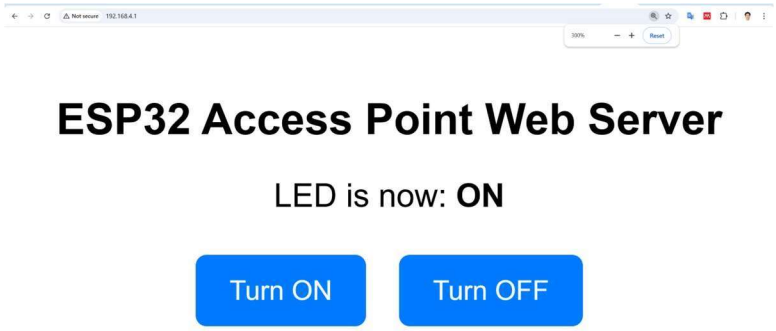
  html += stateText;  // chèn trạng thái LED

  html  +=  R"rawliteral(</strong></p>     <a  href="/on"
class="button">Turn ON</a>
  <a href="/off" class="button">Turn OFF</a>
</body>
</html> )rawliteral";
return html;
}
// Trang chủ void
handleRoot() {
  server.send(200, "text/html", htmlPage());
}
// Bật LED và redirect void
handleOn() {
  digitalWrite(ledPin, HIGH);   ledState = true;
  server.sendHeader("Location", "/", true);   server.send(302, "text/plain", "");
}
// Tắt LED và redirect void
handleOff() {   digitalWrite(ledPin,
LOW);   ledState = false;
  server.sendHeader("Location", "/", true);
```

```
  server.send(302, "text/plain", "");
}
// Redirect mọi đường sai về "/" void
handleNotFound() {
  server.sendHeader("Location", "/", true);   server.send(302, "text/plain", "");
}
void setup() {
  Serial.begin(115200);   pinMode(ledPin,
OUTPUT);   digitalWrite(ledPin, LOW);
WiFi.softAP(ssid, password);
  IPAddress IP = WiFi.softAPIP();
  Serial.print("Access Point IP: ");
  Serial.println(IP);  // thường là 192.168.4.1   server.on("/",
handleRoot);   server.on("/on", handleOn);   server.on("/off",
handleOff);   server.onNotFound(handleNotFound);

  server.begin();
  Serial.println("Web server started.");
}
void loop() {
  server.handleClient();
}
```

Sau khi nạp chương trình thì 1 Wi-Fi có tên ESP32-AP xuất hiện, ta dùng laptop hoặc Điện thoại di động để truy cập và vài trình duyệt để truy cập 192.168.4.1 để vào webserver



Hình 7.9. Webserver điều khiển LED của ESP32

7.2.9 MQTT

MQTT (Message Queuing Telemetry Transport) Là một giao thức gởi dạng publish /subscribe Sử dụng băng thông thấp, độ tin cậy cao và khả năng được sử dụng trong mạng lưới không ổn định.

Eclipse Mosquitto là một MQTT Broker mã nguồn mở, nhẹ, được phát triển trong dự án Eclipse Foundation. Nó cho phép các thiết bị IoT gửi và nhận thông điệp MQTT một cách hiệu quả, theo mô hình publish-subscribe. Server chạy trên Intel Atom N2800. Có thể cài đặt Eclipse Mosquitto tại https://mosquitto.org/download/

Khi đó, trên command promt ta nhập lệnh sau để gửi 1 thông điệp (publish)

```
mosquitto_pub -h localhost -t topic/test -m "on" -q 1
```

Để nhận một thông điệp

```
mosquitto_sub -h localhost -t topic/test -q 1
```

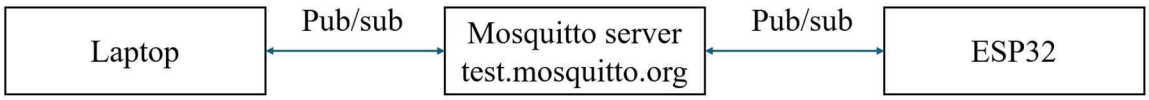
Các tham số trong lệnh MQTT có ý nghĩa như sau:

- -h dùng để chỉ định địa chỉ của broker MQTT, tức là máy chủ trung gian chịu trách nhiệm nhận và phân phối các thông điệp.
- -t xác định tên của topic, đây là kênh thông tin mà các thiết bị sẽ gửi hoặc nhận dữ liệu.
- -m là nội dung của thông điệp cần gửi đi thông qua topic đã chọn.
- -q chỉ mức độ đảm bảo chất lượng dịch vụ truyền (QoS – Quality of Service), với các giá trị 0, 1 hoặc 2 tương ứng với các mức độ đảm bảo khác nhau về việc dữ liệu có được nhận hay không.

Một ví dụ:

```
mosquitto_pub -h example.com -t home/back -m “ok” -q 2
```

Gửi thông điệp "ok" đến topic home/back trên broker tại example.com, với QoS = 2 (đảm bảo tối đa).



Hình 7.10. Sơ đồ Pub/sub trong ví dụ 7.13

Chương trình 7.13. ESP32 MQTT với Mosquitto Broker:
- ESP32 publish “Hello world” trên topic outTopic mỗi 2 giây - ESP32 subscribe trên topic inTopic, nếu nhận được:

```
#include <WiFi.h>
#include <PubSubClient.h> const char* ssid =
"HUTECH"; const char* password =
"HUTECH@123";
const char* mqtt_server = "test.mosquitto.org";
WiFiClient espClient;
PubSubClient client(espClient); unsigned
long lastMsg = 0; #define
MSG_BUFFER_SIZE (50) char
msg[MSG_BUFFER_SIZE]; int value =
0; void setup_wifi() {   delay(10);
Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);   WiFi.begin(ssid,
password);
  while (WiFi.status() != WL_CONNECTED) {     delay(500);
    Serial.print(".");
  }
  randomSeed(micros());
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
}
void callback(char* topic, byte* payload, unsigned int length) {
  Serial.print("Message arrived [");
  Serial.print(topic);   Serial.print("] ");
  for (int i = 0; i < length; i++) {
    Serial.print((char)payload[i]);
  }
  Serial.println();
  if ((char)payload[0] == '0') {
digitalWrite(BUILTIN_LED, LOW);   } else if
((char)payload[0] == '1') {
digitalWrite(BUILTIN_LED, HIGH);
  }
}
void reconnect() {
```

```
  while (!client.connected()) {
    Serial.print("Attempting MQTT connection...");     String
clientId = "ESP32Client-";     clientId += String(random(0xffff),
HEX);     if (client.connect(clientId.c_str())) {
Serial.println("connected");
      client.publish("outTopic", "hello world");       client.subscribe("myInTopic");
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");       delay(5000);
    }
  } }
void setup() {    pinMode(BUILTIN_LED, OUTPUT);  // Initialize the BUILTIN_LED pin as an
output
  Serial.begin(115200);   setup_wifi();
  client.setServer(mqtt_server, 1883);   client.setCallback(callback);
}

void loop() {
  if (!client.connected()) {     reconnect();
  }
  client.loop();
  unsigned long now = millis();   if (now
- lastMsg > 2000) {     lastMsg = now;
++value;
    snprintf(msg, MSG_BUFFER_SIZE, "hello world #%ld", value);
    Serial.print("Publish message: ");     Serial.println(msg);
    client.publish("myOutTopic", msg);
  }
}
```

Mở một command promt tại C:\Program Files\mosquitto rồi nhập lệnh sau để laptop subscribe myOutTOpic

```
.\mosquitto_sub.exe -h test.mosquitto.org -t myOutTopic -q 1
```

Mở một command promt tại C:\Program Files\mosquitto rồi nhập lệnh sau để laptop publish myInTopic cho việc điều khiển đèn LED

```
./mosquitto_pub.exe -h test.mosquitto.org -t myInTopic -q 1 -m "1"
Hoặc
./mosquitto_pub.exe -h test.mosquitto.org -t myInTopic -q 1 -m "0"
```

Sau khi sinh viên đã nắm được cách giao tiếp MQTT thông qua Mosquitto CLI (với các lệnh mosquitto_pub và mosquitto_sub), có thể thấy rằng việc sử dụng dòng lệnh là không trực quan và không dễ theo dõi khi làm việc với nhiều topic hoặc thiết bị cùng lúc. Trong thực tế, có nhiều công cụ đồ họa (GUI) giúp chúng ta quan sát, gửi và nhận dữ liệu MQTT dễ dàng hơn. Những công cụ này không chỉ hỗ trợ học tập hiệu quả mà còn rất phù hợp để triển khai các hệ thống IoT chuyên nghiệp. Một số công cụ GUI phổ biến hỗ trợ MQTT

MQTT Explorer

Là phần mềm phổ biến nhất hiện nay để giám sát MQTT, có giao diện trực quan, hiển thị dạng cây topic, hỗ trợ gửi và nhận dữ liệu, xem lại lịch sử. Rất phù hợp để giảng dạy hoặc test ESP32.

MQTTX (by EMQX)

Là công cụ hiện đại, hỗ trợ nhiều kết nối MQTT cùng lúc. Giao diện thân thiện, hỗ trợ gói tin JSON, WebSocket, MQTT 5.0. Tốt cho kiểm thử hệ thống có nhiều node.

Node-RED

Không chỉ là GUI mà còn là một nền tảng lập trình trực quan cho IoT. Node-RED hỗ trợ kết nối MQTT, thiết kế dashboard điều khiển, vẽ đồ thị cảm biến, lập kịch bản điều khiển... rất thích hợp cho học phần nâng cao hoặc đồ án IoT. HiveMQ Web Client

Là công cụ web miễn phí cho phép publish/subscribe MQTT trực tiếp trên trình duyệt. Không cần cài đặt. Rất nhanh để kiểm tra hoạt động của ESP32 khi gửi dữ liệu lên các broker công cộng như

7.2.10 Node-RED

Node-RED là một công cụ lập trình để kết nối các phần cứng, APIs và các dịch vụ phần cứngĐược xây dựng dựa trên Node.js. Lập trình dựa trên kéo thả. Đầu tiên, ta cần cài đặt Nodejs thông qua https://nodejs.org/en

Kiểm tra phiên bản của Nodejs và npm thông qua gõ lệnh sau trên command promt node --version && npm –version

Kết quả trả về tương tự

```
v22.16.0
10.9.2
```

Cài đặt Node-RED

```
npm install -g --unsafe-perm node-red
```

Ta sẽ cài đặt thêm dashboard cho Node-RED

```
npm i node-red-dashboard
```

Để vận hành Node-RED ta chỉ cần nhập lệnh Node-RED, nếu thành công kết quả sẽ trả về

```
16 Jun 00:39:04 - [info]

Welcome to Node-RED
===================

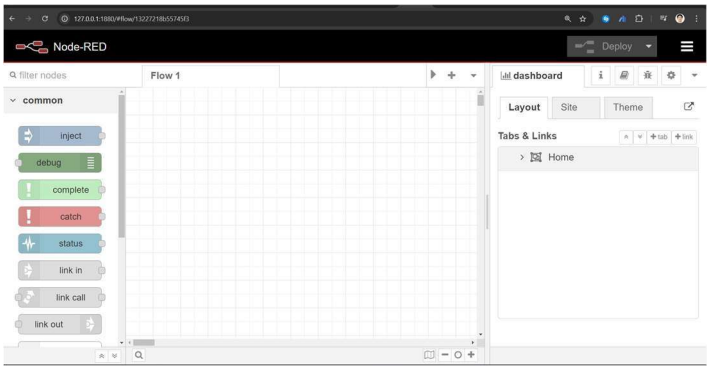
16 Jun 00:39:04 - [info] Node-RED version: v4.0.9
16 Jun 00:39:04 - [info] Node.js  version: v22.16.0
16 Jun 00:39:04 - [info] Windows_NT 10.0.26100 x64 LE
16 Jun 00:39:05 - [info] Loading palette nodes
16 Jun 00:39:05 - [info] Dashboard version 3.6.5 started at /ui
16 Jun 00:39:05 - [info] Settings file  : C:\Users\thuyen\.nodered\settings.js
16 Jun 00:39:05 - [info] Context store  : 'default' [module=memory]
16 Jun 00:39:05 - [info] User directory : C:\Users\thuyen\.node-red
16 Jun 00:39:05 - [warn] Projects disabled : editorTheme.projects.enabled=false
16 Jun 00:39:05 - [info] Flows file     : C:\Users\thuyen\.node-red\flows.json
16 Jun 00:39:05 - [info] Creating new flow file
16 Jun 00:39:05 - [warn]
---------------------------------------------------------------------
Your flow credentials file is encrypted using a system-generated key.
```

If the system-generated key is lost for any reason, your credentials file will not be recoverable, you will have to delete it and re-enter your credentials.
You should set your own key using the 'credentialSecret' option in your settings file. Node-RED will then re-encrypt your credentials file using your chosen key the next time you deploy a change.

---

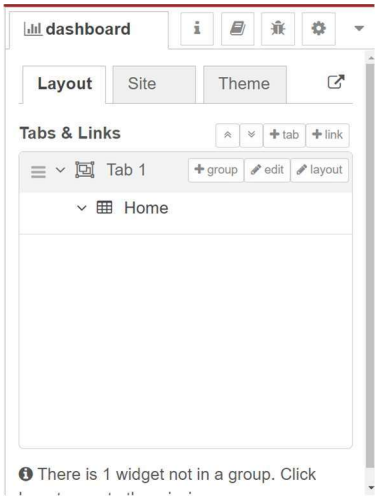
16 Jun 00:39:05 - [info] Server now running at http://127.0.0.1:1880/
16 Jun 00:39:05 - [warn] Encrypted credentials not found
16 Jun 00:39:05 - [info] Starting flows
16 Jun 00:39:05 - [info] Started flows

Ta sẽ truy cập vào 127.0.0.1:1880 để có giao diện Editor



Hình 7.11. Giao diện Editor của Node-RED

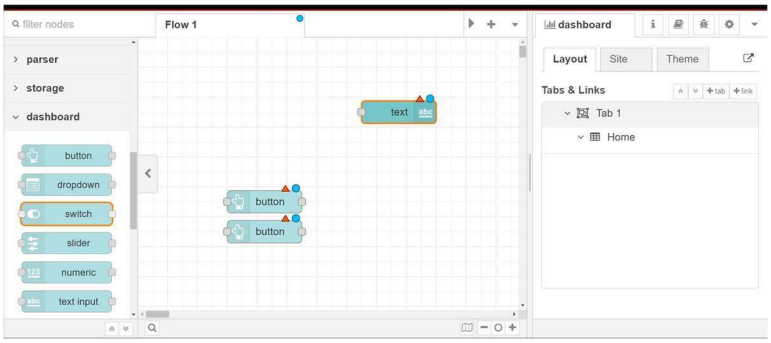
Tại Dashboard -> Layout ta sẽ tạo một Tab với 1 Group đơn giản tên là Home



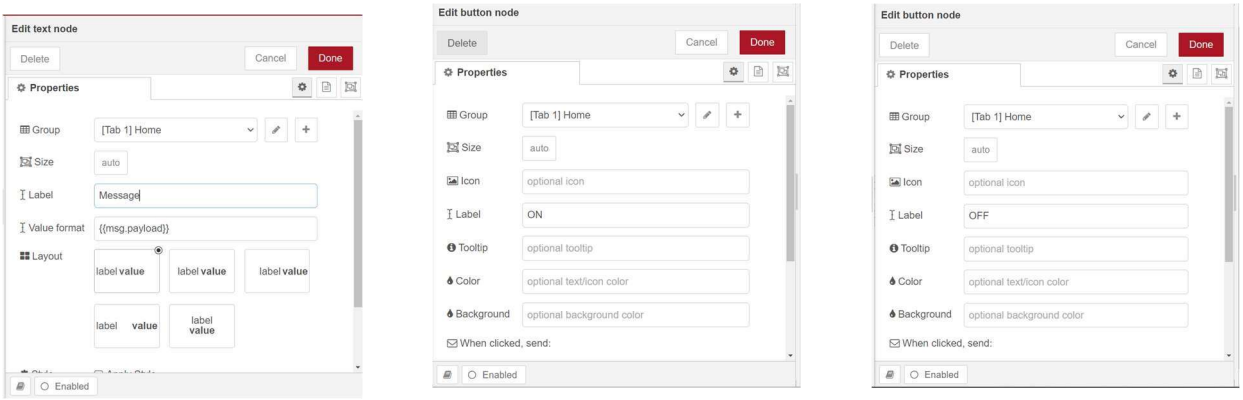
Hình 7.12. Tạo tab với 1 group đơn giản Từ Palette, ta sử dụng:

- ui_button cho nút nhấn ON/OFF đèn

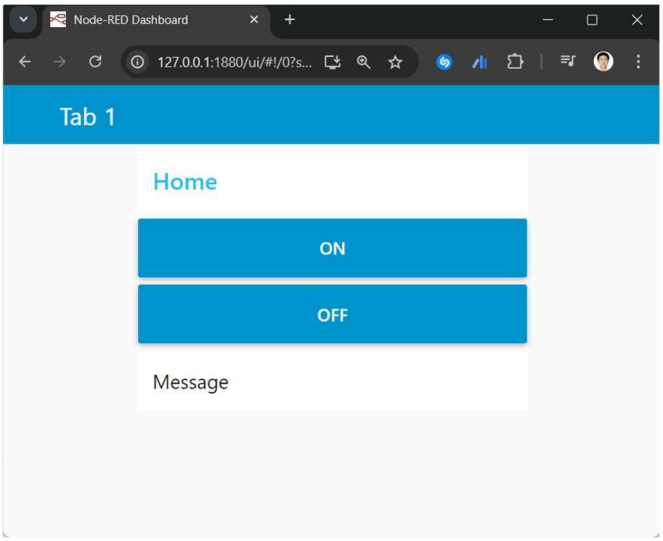
- ui_text cho việc hiển thị message



Hình 7.13. Từ DashBoard, ta chọn ui_button và ui_text

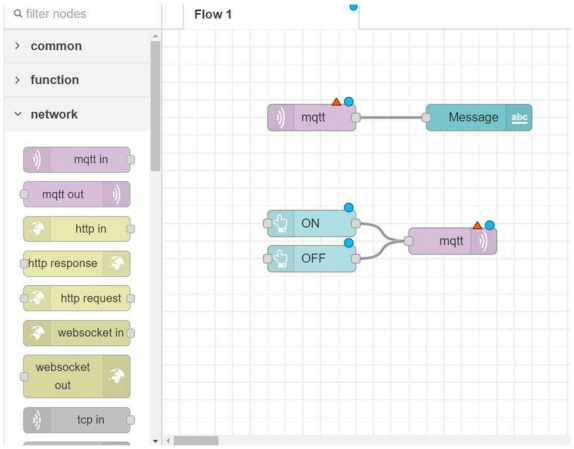


Hình 7.14. Chọn Group cho ui_button và ui_text là [Tab1] Home Khi ta truy cập 127.0.0.1:1880/ui, ta sẽ có giao diện ui



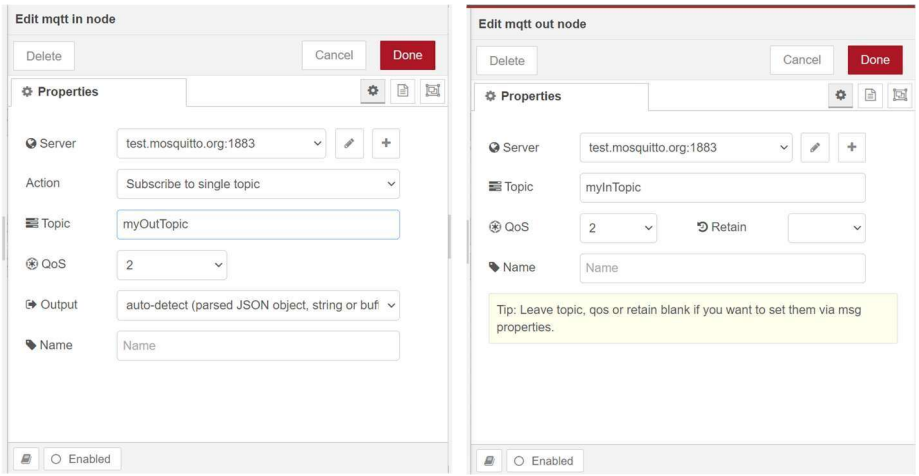
Hình 7.15. Giao diện đã thiết kế bằng Node-RED

Tuy nhiên, ta chỉ có giao diện và chương có logic nào phía dưới cả. trở về 127.0.0.1:1880, ta sẽ thêm mqtt in và mqtt out để subscribe và oublish



Hình 7.16. Sử dụng MQTT trong Node-RED Với các block

được cấu hình như sau:



Hình 7.17. Cấu hình mqtt in và mqtt out

Cuối cùng, ta chọn Deploy, Node-RED ta sẽ hoạt động

TÓM TẮT

Bài 7 giới thiệu toàn diện về vi điều khiển ESP32 – một nền tảng mạnh mẽ tích hợp Wi-Fi và Bluetooth, phổ biến trong các ứng dụng IoT. Nội dung bài học được chia thành hai phần chính:

- Tổng quan phần cứng ESP32: Sinh viên được làm quen với ba dạng phần cứng gồm SoC, Module và DevKit, hiểu rõ sự khác biệt giữa chúng và cách lựa chọn phù hợp với mục đích học tập, thử nghiệm hoặc sản xuất.
- Lập trình ESP32 với Arduino IDE: Bài học hướng dẫn lập trình cơ bản với GPIO, ADC, PWM (LEDC), kết nối Wi-Fi và Bluetooth. Bên cạnh đó, sinh viên còn được tiếp cận lập trình ESP32 như HTTP client/server, MQTT với Mosquitto và tạo giao diện trực quan bằng Node-RED.

Thông qua các ví dụ minh họa cụ thể, bài học giúp sinh viên rèn kỹ năng lập trình ngoại vi, giao tiếp không dây và kết nối mạng trong môi trường IoT. Đây là nền tảng quan trọng cho việc phát triển các hệ thống IoT thực tế trong học phần hoặc đồ án cuối kỳ.

CÂU HỎI ÔN TẬP

Câu 1: ESP32 có mấy dạng phần cứng chính? Hãy phân biệt SoC, Module và DevKit về cấu trúc và ứng dụng.

Câu 2: Lý do vì sao GPIO6 đến GPIO11 không nên sử dụng trong thiết kế mạch với ESP32?

Câu 3: Cho biết sự khác biệt giữa ADC1 và ADC2 trên ESP32. Trong trường hợp nào nên ưu tiên dùng ADC1?

Câu 4: Viết chương trình tạo xung PWM tần số 2500 Hz với độ phân giải 10 bit

Câu 5: Trong một hệ thống IoT sử dụng ESP32 để thu thập dữ liệu nhiệt độ và gửi lên server qua HTTP, giao thức HTTP nào nên được sử dụng: GET hay PUT? Giải thích lựa chọn.

Câu 6: So sánh MQTT và HTTP trong truyền thông IoT. Trường hợp nào nên dùng MQTT thay vì HTTP?

Câu 7: ESP32 có thể hoạt động như một Webserver nội bộ. Mô tả quy trình cơ bản để thiết lập và điều khiển LED qua giao diện web.

Câu 8: Trong hệ thống thu thập dữ liệu nhiều cảm biến dùng ESP32, anh / chị sẽ chọn công cụ nào để giám sát dữ liệu MQTT trực quan? Vì sao?

TÀI LIỆU THAM KHẢO

1. Rolando Herrero (2022), Fundamentals of IoT Communication Technologies, Springer Nature Switzerland AG
2. Buyya và Dastjerdi (2016), Internet of Things Principles and Paradigms, Elsevier
3. Hanes và các cộng sự (2017), IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things